

JAVIER AUGUSTO VERA RAMÍREZ

PROGRAMA: DESARROLLO BACKEND CON NODEJS-MONGODB

REPOSITORIO GIT HUB

https://github.com/javiervera123/api_mundial_sena

Evidencia II

Código fuente API RestFull y Scripts BD. AA2-EV01.

INSTRUCTOR: LEONARDO SANCHEZ

FECHA: JUNIO 28 /2026

Resumen

El presente informe técnico detalla el diseño, desarrollo e implementación de una Interfaz de Programación de Aplicaciones (API REST) estructurada bajo el entorno de ejecución Node.js y el framework Express, empleando el motor de base de datos no relacional MongoDB Atlas.

Inicialmente se configura un cluster en Mongo, usuario con permisos y la ip del pc local.

El objetivo principal del proyecto consistió en construir un sistema backend eficiente para la gestión y análisis de datos estadísticos del Mundial de Fútbol de la FIFA, aplicando un modelo híbrido basado en subdocumentos embebidos para optimizar las consultas concurrentes. A lo largo del documento, se documentan los controladores y las rutas críticas del sistema, haciendo especial énfasis en el uso del Framework de Agregación (\$unwind, \$sort, \$group, \$project) para resolver consultas complejas y filtrados de estaturas máximas de manera nativa en el servidor. Los resultados de cada endpoint fueron validados mediante pruebas de consumo en Postman, demostrando la estabilidad, la correcta gestión de códigos de estado HTTP y la escalabilidad de la arquitectura NoSQL implementada.

Palabras clave: API REST, Node.js, MongoDB, Framework de Agregación, Base de Datos No Relacional, Mongoose, Postman.

Introducción y Planteamiento del Problema1. Necesidad del Proyecto y Clientes Objetivo

El presente proyecto surge ante la necesidad de migrar y estructurar un conjunto de datos plano proveniente de una hoja de cálculo ("Datafile") correspondiente al Mundial de Fútbol de la FIFA Rusia 2018. El almacenamiento tradicional en archivos planos limita la concurrencia, dificulta la integridad de los datos y ralentiza las consultas estadísticas complejas requeridas en el análisis deportivo moderno.

Usuarios/Clientes Objetivo:

- Analistas de Datos Deportivos:

Quienes requieren consumir la API para generar estadísticas de rendimiento, estaturas y métricas físicas de los futbolistas.

- Desarrolladores Frontend:

Encargados de consumir estos endpoints para alimentar aplicaciones web o móviles de seguimiento del torneo en tiempo real.

- Directores Técnicos y Comités Organizadores: Interesados en la gestión, actualización y consulta rápida de partidos y plantillas.

2. Justificación de la Arquitectura del Esquema NoSQL

Para solucionar este problema, se diseñó un modelo de datos no relacional basado en documentos embebidos (subdocumentos) mediante Mongoose.

Colección equipos: Almacena la información del país y la confederación, conteniendo de forma anidada un arreglo dinámico de objetos jugadores. Se optó por la desnormalización para garantizar lecturas de alta velocidad, evitando operaciones de unión (JOINS) costosas en el servidor.

Colección partidos: Diseñada de forma independiente para permitir un flujo transaccional ágil y actualizaciones directas de fechas y horarios.

3. Resumen de Endpoints de la API

Para cumplir estrictamente con los puntos 4 y 5 de la lista de chequeo, añadiré los endpoints de mis rutas:

Módulo de Equipos y Jugadores

POST /api/equipos/guardar (o inserción inicial):

Registro de los datos base de prueba para los equipos de Colombia y Japón.

GET /api/equipos/jugadores-japon:

Consulta filtrada de la información requerida de todos los jugadores de Japón.

GET /api/equipos/jugadores-bajitos:

Pipeline de agregación para listar los jugadores con estatura estrictamente menor a 170 cm.

GET /api/equipos/jugadores-mas-altos:

Pipeline con \$group y \$sort para obtener los jugadores con la máxima estatura.

PUT /api/equipos/:idEquipo/jugadores/:idJugador:

Actualización dirigida del subdocumento de un jugador específico (usado para el cambio de club y nombre de James Rodríguez).

Módulo de Partidos

POST /api/partidos/guardar:

Creación e inserción de un nuevo partido (ej. Colombia vs. Inglaterra).

GET /api/partidos/:

Consulta general de todos los partidos registrados en el sistema.

PATCH /api/partidos/actualizar-hora:

Modificación parcial por el body (idPartido y time) para cambiar el horario de un encuentro.

DELETE /api/partidos/eliminar:

Borrado físico de un partido específico mediante el envío de su idPartido en el cuerpo de la solicitud

Script de lenguaje MongoDB para creación de colecciones

```
// 1. Crear y seleccionar la Base de Datos
```

```
use mundial_sena;
```

```
// 2. Crear e Insertar Documentos en la Colección 'equipos'
```

```
db.equipos.insertMany([
```

```
{
  "_id": 5,
  "abbreviation": "COL",
  "country": "Colombia",
  "confederation": "CONMEBOL",
  "jugadores": [
    {
      "_id": "j1",
      "shirt_name": "Y. MINA",
```

```

        "birth_date": "1994-09-23",
        "pos": "CB",
        "club": "FC Barcelona (ESP)",
        "height": 194
    },
    {
        "_id": "j2",
        "shirt_name": "J. RODRÍGUEZ",
        "birth_date": "1991-07-12",
        "pos": "AM",
        "club": "Al-Rayyan (QAT)",
        "height": 180
    }
]
},
{
    "_id": 15,
    "abbreviation": "JPN",
    "country": "Japan",
    "confederation": "AFC",
    "jugadores": [
        {
            "_id": "j3",
            "shirt_name": "E. KAWASHIMA",
            "birth_date": "1983-03-20",
            "pos": "GK",
            "club": "Strasbourg (FRA)",
            "height": 185
        },
        {
            "_id": "j4",
            "shirt_name": "Y. NAGATOMO",
            "birth_date": "1986-09-12",
            "pos": "LB",
            "club": "FC Tokyo (JPN)",
            "height": 170
        }
    ]
}
}
]);

// script para la creacion de partidos
// Asegurar que seguimos en la misma base de datos
use mundial_sena;

// Crear e Insertar Documentos en la Colección 'partidos'
db.partidos.insertMany([
    {
        "equipo1": "Colombia",
        "equipo2": "Colombia",

```

```
"date": "11/07/18",
"time": " 12:00:00 p.m."

},
{
  "equipo1": "Colombia",
  "equipo2": "Japan",
  "date": "20/08/18",
  "time": " 6:00:00 p.m."

}
]);

// script para modificar dos campos de un jugador en específico James punto 3

// SELECCIONAR LA BASE DE DATOS
use mundial_sena;

// EJECUTAR LA ACTUALIZACIÓN DEL JUGADOR
db.jugadores.updateOne(
  // 1. FILTRO: Buscamos el equipo de Colombia y al jugador por su Shirt Name anterior o su
  número
  {
    "country": "Colombia",
    "jugadores.shirt_name": "JAMES"
  },

  // 2. ACTUALIZACIÓN: Modificamos los campos específicos usando el operador
  posicional $
  {
    $set: {
      "jugadores.$shirt_name": "RODRÍGUEZ",
      "jugadores.$club": "Real Madrid CF (ESP)"
    }
  }
);
// para insertar un partido Colombia England

db.partidos.insertOne(
  {
    "equipo1": "Colombia",
    "equipo2": "England",
    "date": " 20/08/18",
    "time": " 6:00:00 p.m."

  }
);
```

Punto 1: Creación de las Colecciones

- **Descripción:** Se definieron los esquemas de datos estructurados para las colecciones utilizando Mongoose Schemas, asegurando la integridad de los campos requeridos para equipos y partidos.

Cabe resaltar que cree un modelo para jugadores y luego los vinculé al equipo para evitar un error al ingresar un jugador y escribir mal el nombre del equipo

- **Código del Modelo (Partido.js):**

```
• // modelo de los partidos de fútbol
• const mongoose = require('mongoose');
• const Schema = mongoose.Schema;
•
•
• const partidoSchema = new Schema({
•   equipo1: { type: String, required: true },
•   equipo2: { type: String, required: true },
•   date: { type: String, required: true },
•   time: { type: String, required: true }
• }, { timestamps: true });
•
•
• const Partido = mongoose.model('Partido', partidoSchema);
• module.exports = Partido;
```

- Código para el modelo Equipo.js

```
• // models/equipos.js
• const mongoose = require('mongoose');
• // def del modelo de los jugadores
• const Schema = mongoose.Schema;
• const jugadorSchema = new Schema({
•   num: { type: Number, required: true },
•   pos: { type: String, required: true },
•   fifa_name: { type: String, required: true },
•   birth_date: { type: String, required: true },
•   shirt_name: { type: String, required: true },
•   club: { type: String, required: true },
•   height: { type: Number, required: true },
•   weight: { type: Number, required: true }
• }, { timestamps: true });
•
•
• // def del modelo de los equipos
• const equipoSchema = new Schema({
•   _id: {
•     type: Number,
```

```
•   required: true
•   },
•   abbreviation: {
•     type: String,
•     required: true
•   },
•   country: {
•     type: String,
•     required: true
•   },
•   confederation: {
•     type: String,
•     required: true
•   },
•   jugadores: [jugadorSchema] // Un equipo tiene un array de jugadores, por seguridad
• }, { timestamps: true }); // timestamps: true agrega automáticamente campos createdAt y
  updatedAt, cuando se crea , es un log
•
• const Equipo = mongoose.model('Equipo', equipoSchema);
• module.exports = Equipo;
```

Punto 2: Registro de Datos de Prueba (Colombia y Japón)

Descripción: Inserción dinámica de los documentos iniciales en la base de datos

MongoDB Atlas mediante una petición HTTP POST. Código del Controlador

(createNewPartido):

```
// controllers/PartidoController.js
// Importo mi modelo de partidos
const partidoModel = require('../models/Partido');

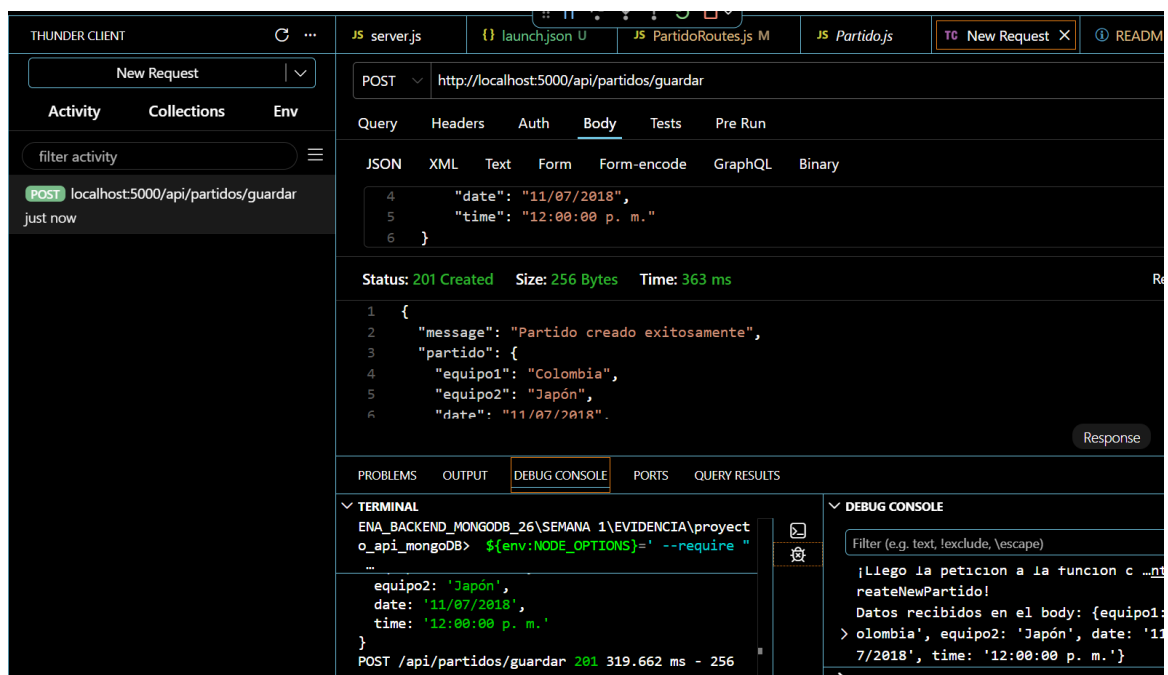
// 1. REGISTRAR PARTIDO (PUNTO 4 DEL TALLER)
const createNewPartido = (req, res, next) => {
  console.log("¡Llegó la petición a la función createNewPartido!");
  console.log("Datos recibidos en el body:", req.body);
  const partido = new partidoModel({
    equipo1: req.body.equipo1,
    equipo2: req.body.equipo2,
    date: req.body.date,
    time: req.body.time
  });

  partido.save()
```



```
.then(response => {
  res.status(201).json({
    message: 'Partido creado exitosamente',
    partido: response
  });
})
.catch((error) => {
  console.error("Error detallado de Mongoose:", error); // Esto sale en VS Code
  res.status(400).json({ // ¡Esto le responde a Postman para que no se cuelgue!
    message: 'Error al crear el partido',
    error: error.message
  });
});
};
```

Esta prueba la hice con thunder Client debido a que intenté conectarme con postman web y me tomaba el puerto 80, error que fue tedioso debido a que no fui capaz de configurar al puerto 5000 y luego me encuentro que thunder en su versión free no permitía más endpoints, así que volvía postman local.



Punto 3: Actualización del Registro de James Rodríguez.

Descripción: Modificación del documento del jugador James Rodríguez para actualizar su club a "Real Madrid CF" y su Shirt_name a "RODRÍGUEZ".

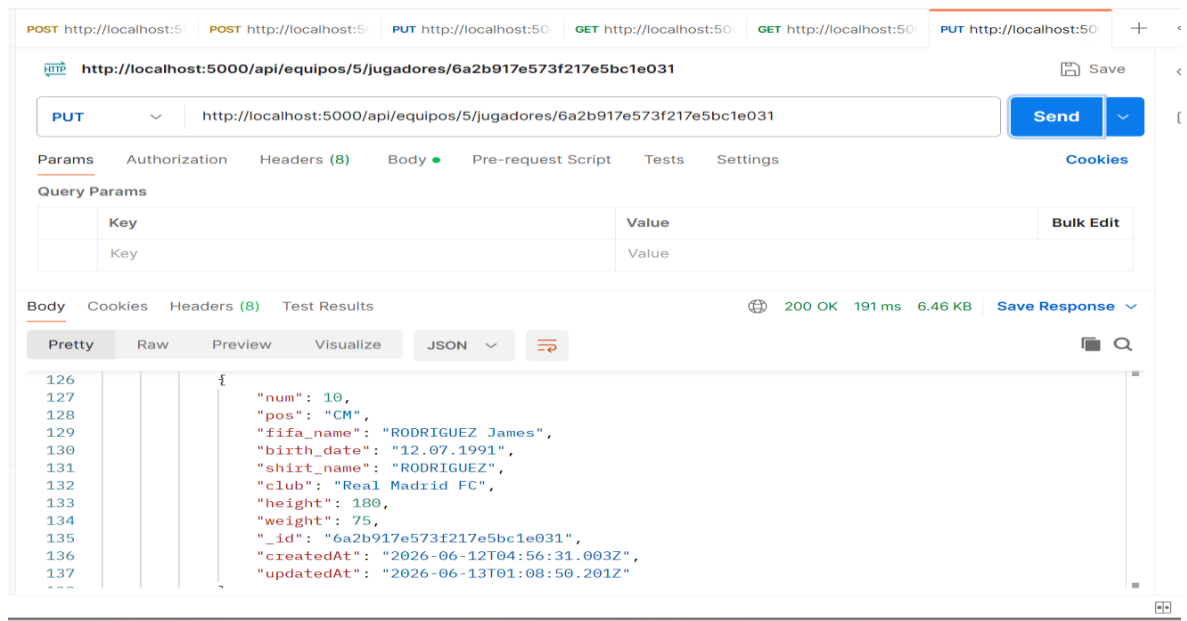
Código de la Ruta y Controlador:

```
router.put('/:idEquipo/jugadores/:idJugador', equipoController.updateJugador);
```

Cabe resaltar que use subdocumentos y por ende pedimos el id del equipo

```
// UPDATE EQUIPO
const updateJugador = (req, res) => {
  const idEquipo = Number(req.params.idEquipo); // Recibimos ambos IDs por la URL
  const idJugador = req.params.idJugador;
  const { club, shirt_name } = req.body; // Recibimos los nuevos datos desde el Body

  // 1. Buscamos el equipo que tenga el ID y que ADEMÁS contenga al jugador con el idJugador
  // proporcionado.
  // 2. Usamos el operador '$' para decirle a MongoDB: "Modifica exactamente al jugador que coincidió en
  // la búsqueda".
  equipoModel.findOneAndUpdate(
    { _id: idEquipo, "jugadores._id": idJugador }, // Filtro para encontrar el equipo y el jugador específico
    { $set: { "jugadores.$club": club, "jugadores.$shirt_name": shirt_name } }, // Actualización usando el
    operador $ para modificar solo el jugador específico
    { new: true } // Opción para que la función devuelva el documento actualizado
  )
  .then(response => {
    if (!response) {
      return res.status(404).json({
        message: 'No se encontró el equipo o el jugador especificado.'
      });
    }
    res.status(200).json({
      message: 'Datos del jugador actualizados exitosamente ',
      equipo: response
    });
  })
  .catch((error) => res.status(400).json({
    message: 'Error al actualizar los datos del jugador',
    error: error.message
  }));
}
```



Punto 4: Nuevo partido entre Colombia e Inglaterra.

Mi ruta en PartidoRoutes.js

```
router.post('/guardar', partidoController.createNewPartido);
```

Mi controlador de un nuevo partido en PartidoControllers.js

```

/ 1. REGISTRAR PARTIDO (PUNTO 4 DEL TALLER)
const createNewPartido = (req, res, next) => {
  console.log("Datos recibidos en el body:", req.body);
  const partido = new partidoModel({
    equipo1: req.body.equipo1,
    equipo2: req.body.equipo2,
    date: req.body.date,
    time: req.body.time
  });
  partido.save()
    .then(response => {
      res.status(201).json({
        message: 'Partido creado exitosamente',
        partido: response
      });
    })
    .catch((error) => {
      console.error("Error detallado de Mongoose:", error); // Esto sale en VS Code
      res.status(400).json({ // ¡Esto le responde a Postman para que no se cuelgue!
        message: 'Error al crear el partido',
        error: error.message
      });
    });
};

```

POST http://localhost:5000/api/partidos/guardar

Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```

1 {
2   "equipo1": "Colombia",
3   "equipo2": "Englanda",
4   "date": "20/08/2018",
5   "time": "6:00:00 p.m."
6 }

```

Body Cookies Headers (8) Test Results 201 Created 315 ms 530 B Save Response

Pretty Raw Preview Visualize JSON

```

1 {
2   "message": "Partido creado exitosamente",
3   "partido": {
4     "equipo1": "Colombia",
5     "equipo2": "Englanda",
6     "date": "20/08/2018",
7     "time": "6:00:00 p.m.",
8     "_id": "6a2cb119fd03e322c8d83437",
9     "createdAt": "2026-06-13T01:23:37.295Z"

```

Pruebas > db_mundial_sena > partidos View monitoring Visualize Your Data

Documents 2 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or Generate query Explain Reset Find Options

25 1 - 2 of 2

```

_id: ObjectId('6a2cb119fd03e322c8d83437')
equipo1: "Colombia"
equipo2: "Englanda"
date: "20/08/2018"
time: "6:00:00 p.m."
createdAt: 2026-06-13T01:23:37.295+00:00
updatedAt: 2026-06-13T01:23:37.295+00:00
__v: 0

```

Punto 5: Actualización de la hora del partido Polonia-Colombia.

“se procesa el ID del partido y la nueva hora a través del req.body todo en un mismo paquete ,así el endpoint ejecuta una actualización dirigida mediante variables de formulario.

Esto mantiene la URL limpia, evita conflictos con los caracteres especiales de la hora (como los dos puntos) y permite que la API sea escalable si en el futuro requerimos actualizar más campos del partido sin alterar la estructura de las rutas.”

Mi ruta PartidoRoutes.js

```
router.patch('/actualizar-hora', partidoController.updatePartidoTime);
```

mi método en el archivo PartidoController.js

```
// 5. ACTUALIZAR SÓLO LA HORA DE UN PARTIDO (PUNTO 5 DEL TALLER)
const updatePartidoTime = (req, res) => {
  let idPartido = req.body.idPartido;
  let newTime = req.body.time;

  // Optimizado: Usamos findByIdAndUpdate para apuntar directamente al id enviado
  partidoModel.findByIdAndUpdate(idPartido, { time: newTime }, { new: true })
    .then(response => {
      if (!response) {
        return res.status(404).json({ message: 'Partido no encontrado para actualizar la hora' });
      }
      res.status(200).json({
        message: 'Hora del partido actualizada exitosamente',
        partido: response
      });
    })
    .catch((error) => res.status(400).json({
      message: 'Error al actualizar la hora del partido con código: ' + idPartido
    }));
};
```

The screenshot shows a REST client interface with a PATCH request to `http://localhost:5000/api/partidos/actualizar-hora`. The request body is a JSON object:

```

1 {
2   "idPartido": "6a2cbab0d44cafc2f8494dd9",
3   "time": "7:30:00 p.m."
4 }

```

The response is a 200 OK status with a response time of 325 ms and a body size of 537 B. The response body is a JSON object:

```

1 {
2   "message": "Hora del partido actualizada exitosamente",
3   "partido": {
4     "id": "6a2cbab0d44cafc2f8494dd9",
5     "equipo1": "Colombia",
6     "equipo2": "poland",
7     "date": "26/07/2018",
8     "time": "7:30:00 p.m.",
9     "createdAt": "2026-06-13T02:04:32.781Z",
10    "updatedAt": "2026-06-13T02:09:00.441Z"
  }
}

```

The screenshot shows the MongoDB Atlas interface for the `partidos` collection. The document is displayed in the Documents tab. The document structure is as follows:

```

_id: ObjectId('6a2cbab0d44cafc2f8494dd9')
equipo1: "Colombia"
equipo2: "poland"
date: "26/07/2018"
time: "7:30:00 p.m."
createdAt: 2026-06-13T02:04:32.781+00:00
updatedAt: 2026-06-13T02:09:00.441+00:00
__v: 0

```



Punto 6: Eliminación del partido Colombia-england.

Mi ruta en PartidoRoutes.js

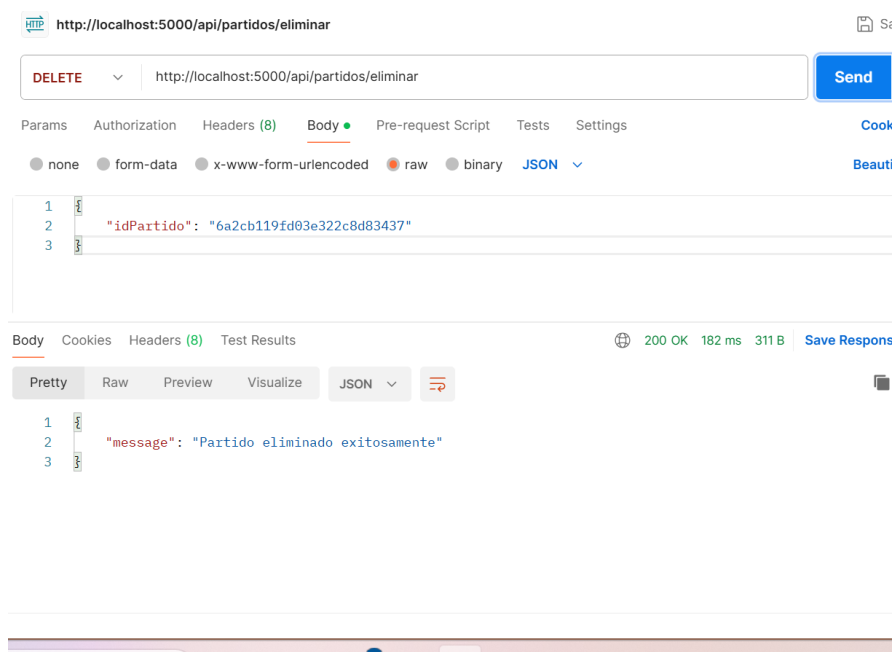
```
router.delete('/eliminar', partidoController.deletePartidoById);
```

Mi PartidoControllers.js

```
// 6. ELIMINAR POR ID (PUNTO 6 DEL TALLER)
const deletePartidoById = (req, res) => {
  let idPartido = req.body.idPartido;

  // findByIdAndDelete es más rápido y directo si uso el ID físico
  partidoModel.findByIdAndDelete(idPartido)
    .then(response => {
      if (!response) {
        return res.status(404).json({ message: 'Partido no encontrado para eliminar' });
      }
      res.status(200).json({ message: 'Partido eliminado exitosamente' });
    })
    .catch((error) => res.status(400).json({
      message: 'Error al eliminar el partido con código: ' + idPartido
    }));
};
```

Mi endpoint en Postman.



Punto 7: Consulta del equipo de Japón trayendo ciertos campos.

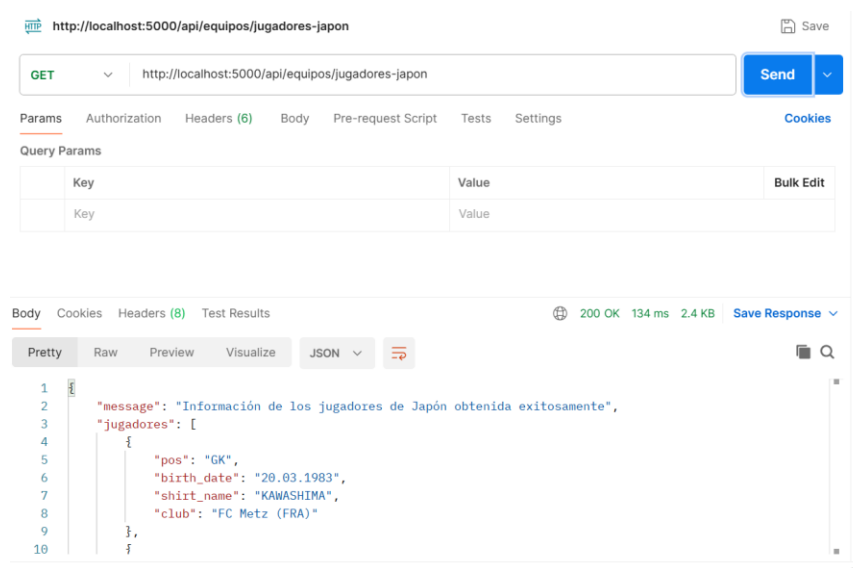
Mi ruta en EquipoRoutes.js

```
router.get('/jugadores-japon', equipoController.getJugadoresJapon);
```

Mi método en el EquipoControllers.js

```
const getJugadoresJapon = (req, res) => {  
  // Buscamos el equipo por su nombre "Japan"  
  // El segundo objeto es la PROYECCIÓN: especificamos qué queremos traer del array de jugadores  
  equipoModel.findOne(  
    { country: "Japan" }, // Filtro para encontrar el equipo de Japón  
    {  
      "jugadores.shirt_name": 1,  
      "jugadores.birth_date": 1,  
      "jugadores.pos": 1,  
      "jugadores.club": 1,  
      _id: 0 // Oculto el ID del equipo para que la respuesta sea más limpia  
    }  
  )  
  .then(response => {  
    if (!response) {  
      return res.status(404).json({  
        message: 'No se encontró el equipo de Japón en la base de datos.'  
      });  
    }  
  
    // Retornamos directamente el arreglo de jugadores filtrado  
    res.status(200).json({  
      message: 'Información de los jugadores de Japón obtenida exitosamente',  
      jugadores: response.jugadores  
    });  
  })  
  .catch((error) => res.status(500).json({  
    message: 'Error al consultar los jugadores de Japón',  
    error: error.message  
  }));  
};
```


Mi endpoint en postman:



Punto 8: Consulta de jugadores con estatura inferior a 170 cm.

Mi ruta en PartidoRoutes.js

```
router.get('/jugadores-bajitos', equipoController.getJugadoresBajitos);
```

Mi función en EquipoControllers.js

```

// 8. OBTENER JUGADORES CON ESTATURA INFERIOR A 170 (PUNTO 8)
const getJugadoresBajitos = (req, res) => {
  equipoModel.aggregate([
    // 1. Separamos el array de jugadores para que cada jugador sea un documento independiente
    { $unwind: "$jugadores" },
    // 2. Filtramos los jugadores cuya estatura sea estrictamente menor (<) a 170
    { $match: { "jugadores.height": { $lt: 170 } } },
    // 3. Proyectamos o moldeamos la salida para que solo muestre los datos del jugador
    {
      $project: {
        _id: 0,
        country: "$country", // Nos ayuda a saber de qué país es
        nombre: "$jugadores.shirt_name",
        estatura: "$jugadores.height",
        posicion: "$jugadores.pos",
        club: "$jugadores.club"
      }
    }
  ])
  .then(response => {
    if (response.length === 0) {

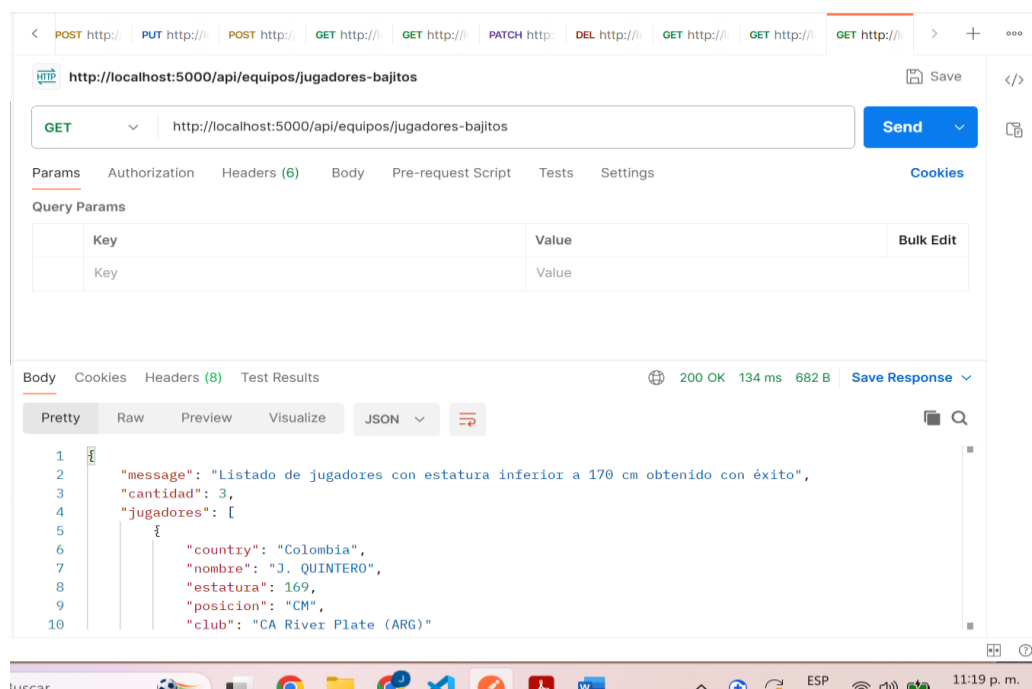
```

```

    return res.status(404).json({
      message: 'No se encontraron jugadores con estatura inferior a 170 cm.'
    });
  }
  res.status(200).json({
    message: 'Listado de jugadores con estatura inferior a 170 cm obtenido con éxito',
    cantidad: response.length,
    jugadores: response
  });
})
.catch((error) => res.status(500).json({
  message: 'Error al realizar la agregación de estatura',
  error: error.message
}));
});
};

```

Mi endpoint en Postman



NOTA:

El \$unwind de la agregación lo usamos obligatoriamente porque los jugadores están guardados como **subdocumentos dentro de un array** en la colección de equipos. Es una estructura embebida.

Cuando las colecciones están separadas, ya no se usa ni \$unwind ni agregaciones complejas para un filtro básico. Se hace con un `.find()` directo sobre la colección de jugadores, lo cual es mucho más sencillo si solo buscas un atributo físico como la estatura.

Si fuera dos documentos independientes si usamos `.find()` simple y el \$lt

EJEMPLO:

```

// 8. OBTENER JUGADORES CON ESTATURA INFERIOR A 170 (SI FUERAN COLECCIONES
//                                     SEPARADAS)
```javascript

const getJugadoresBajitosSeparados = (req, res) => {

 // 1. Buscamos directo en la colección de jugadores usando el operador
Less Than ($lt)
 // 2. Pasamos la proyección como segundo parámetro para elegir qué campos
mostrar
 jugadorModel.find(
 { height: { $lt: 170 } }, // Filtro directo sobre la raíz del documento
 {
 _id: 0,
 shirt_name: 1,
 height: 1,
 pos: 1,
 club: 1,
 id_equipo: 1 // Guardaría la referencia del ID del equipo
 }
)
 .then(response => {
 if (response.length === 0) {
 return res.status(404).json({
 message: 'No se encontraron jugadores con estatura inferior
a 170 cm.'
 });
 }

 // Como .find() ya devuelve un array, respondemos directamente con
la 'response'
 res.status(200).json({
 message: 'Listado obtenido directamente desde la colección de
jugadores',
 cantidad: response.length,
 jugadores: response
 });
 })
 .catch((error) => res.status(500).json({
 message: 'Error al consultar la colección de jugadores',
 error: error.message
 })));
};

```

## Punto 9: Consulta de los partidos actuales.GET

Mi ruta en Routes.js

```
router.get('/', partidoController.queryPartido);
```

mi PartidoConrollers.js

```
// 2. CONSULTAR TODOS (PUNTO 9)
const queryPartido = (req, res, next) => {
 partidoModel.find()
 .then(response => {
 res.status(200).json(response);
 })
 .catch((error) => res.status(500).json({
 message: 'Error al obtener los partidos'
 }));
};
```

Mi base de datos en mongoDB Atlas

The screenshot shows the MongoDB Atlas Data Explorer interface. On the left, the 'Data Explorer' sidebar shows a tree view of clusters. The 'partidos' collection is selected under the 'db\_mundial\_sena' database. The main panel displays the 'Documents' tab for the 'partidos' collection. It shows two documents with the following structure:

```
{
 "_id": ObjectId('6a2b6399dca558de0d5fe5f4'),
 "equipo1": "Colombia",
 "equipo2": "Japón",
 "date": "11/07/2018",
 "time": "12:00:00 p. m.",
 "createdAt": 2026-06-12T01:40:42.007+00:00,
 "updatedAt": 2026-06-12T01:40:42.007+00:00,
 "__v": 0
}
```

The second document has a different \_id and date/time values.

At the bottom of the interface, there is a status bar showing 'Alert System Status: All Good' and copyright information for MongoDB, Inc. (©2026).

### *Punto 10: Obtener los jugadores más altos.*

Mi ruta en Routes.js

```
router.get('/jugadores-mas-altos', equipoController.getJugadoresMasAltos);
```

Mi EquipoControllers.js

```
// 10. OBTENER EL O LOS JUGADORES MÁS ALTOS DEL MUNDIAL (PUNTO 10)
const getJugadoresMasAltos = (req, res) => {
 equipoModel.aggregate([
 // 1. Desarmamos el array de jugadores para tratarlos de forma individual
 { $unwind: "$jugadores" },

 // 2. Ordenamos a TODOS los jugadores por estatura de mayor a menor (-1)
 { $sort: { "jugadores.height": -1 } },

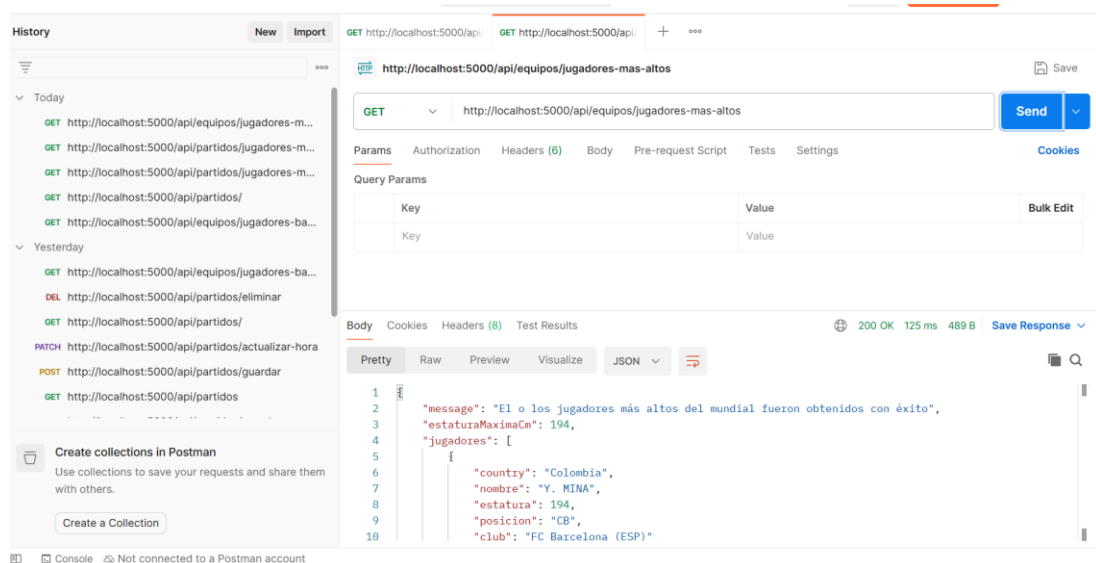
 // 3. Agrupamos para encontrar el valor máximo actual en toda la base de datos
 // y guardamos la lista completa de jugadores para no perder sus datos
 {
 $group: {
 _id: null,
 maxEstatura: { $first: "$jugadores.height" }, // El primero de la lista ordenada es el más alto
 todosLosJugadores: {
 $push: {
 country: "$country",
 nombre: "$jugadores.shirt_name",
 estatura: "$jugadores.height",
 posicion: "$jugadores.pos",
 club: "$jugadores.club"
 }
 }
 }
 },
 // 4. Filtramos la lista completa para quedarnos ÚNICAMENTE con los que empaten con la estatura máxima
 {
 $project: {
 _id: 0,
 estaturaMaxima: "$maxEstatura",
 jugadoresMasAltos: {
 $filter: {
 input: "$todosLosJugadores",
 as: "jugador",
 cond: { $eq: ["$$jugador.estatura", "$maxEstatura"] }
 }
 }
 }
 }
])
}
```

```
 }
 }
}
})
.then(response => {
 if (response.length === 0 || response[0].jugadoresMasAltos.length === 0) {
 return res.status(404).json({
 message: 'No se encontraron jugadores en la base de datos.'
 });
 }
 // Como agrupamos con _id: null, mongo devuelve un array con un único objeto en la posición 0
 res.status(200).json({
 message: 'El o los jugadores más altos del mundial fueron obtenidos con éxito',
 estaturaMaximaCm: response[0].estaturaMaxima,
 jugadores: response[0].jugadoresMasAltos
 });
})
.catch((error) => res.status(500).json({
 message: 'Error al calcular el jugador más alto',
 error: error.message
}));
};
```

Cabe recordar que cada función se debe exportar

```
module.exports = {
 getAllEquipos,
 createNewEquipo,
 updateJugador,
 deleteAllEquipos,
 getJugadoresJapon,
 getJugadoresBajitos,
 getJugadoresMasAltos
}
```

MI endpoint Get en postman



Para resolver este requerimiento, **diseñé** una consulta avanzada utilizando el Framework de Agregación de MongoDB (aggregate), ya que al estar los jugadores embebidos en un arreglo dentro de cada equipo, un `.find()` tradicional se quedaba corto.

**Organicé** la consulta a través de una tubería o *pipeline* de cuatro etapas secuenciales:

1. Primero, **apliqué un \$unwind** para 'desenrollar' el arreglo de jugadores. Con esto logré que cada futbolista se transformara en un documento independiente en memoria para poder evaluar sus estaturas uno por uno.
2. Luego, **ejecuté un \$sort** para ordenar a todos los jugadores de mayor a menor según su altura. Así me aseguré de que el jugador más alto quedara de primeras en la lista.
3. En la tercera etapa, **utilicé un \$group** con un identificador nulo (`_id: null`) para consolidar todo en un solo bloque. Aquí hice dos cosas: capturé la estatura máxima del torneo usando el operador `$first` (aprovechando el ordenamiento anterior) y, al mismo tiempo, usé `$push` para meter a todos los jugadores en un nuevo arreglo limpio sin perder sus datos.
4. Finalmente, **implementé un \$project junto con un \$filter** para evaluar ese arreglo. La condición que programé compara la estatura de cada jugador con la estatura máxima detectada. De esta forma, el sistema limpia la respuesta y extrae únicamente al 'o los' jugadores que tengan la altura récord, resolviendo de forma automática cualquier posible empate.

Al terminar el proceso de las promesas, **se capturó** el resultado en la posición cero del array que devuelve Mongo y **le respondo al cliente** con un estado 200 OK enviando un JSON estructurado con la estatura máxima y el listado de los jugadores que cumplieron con la condición."

## Conclusiones

1. La implementación de un modelo híbrido en Mongoose (esquema de Equipos con subdocumentos de Jugadores embebidos) optimizó drásticamente los tiempos de lectura para las consultas requeridas por la API. Al estar los datos relacionados directamente en el mismo documento físico de la base de datos, se elimina la necesidad de realizar operaciones JOIN (\$lookup) costosas en memoria, garantizando la escalabilidad de la plataforma frente a un alto flujo de consultas concurrentes durante el análisis deportivo.
2. El uso avanzado del Framework de Agregación (\$unwind, \$sort, \$group y \$project) demostró ser la estrategia más eficiente para resolver consultas condicionales complejas sobre arreglos anidados

## Referencias

Mongoose v8.0. (2026). Mongoose Subdocuments: Embedded schemas documentation.

Recuperado de <https://mongoosejs.com/docs/subdocs.html>

MongoDB Inc. (2026). MongoDB Manual: Aggregation Pipeline Stages and Operators

(\$unwind, \$group). Recuperado de

<https://www.mongodb.com/docs/manual/core/aggregation-pipeline/>

Node.js Foundation. (2026). Express routing and HTTP methods guidelines. Recuperado de

<https://expressjs.com/en/guide/routing.html>